# Web Launch Guard — Production Launch Manual

Step-by-step guide to take this codebase from local development to a working production deployment.

The flow assumes you are deploying to Netlify, with Supabase as the database and auth provider, Stripe for billing, and Anthropic for the AI analysis.

A note on order: you cannot fully configure Stripe webhooks or Supabase OAuth redirects until you have a public deployment URL. The phases below follow the real chicken-and-egg sequence: provision external services first, do an initial deploy to get a URL, then circle back to wire webhooks and redirects.

---

## Phase 0 — Prerequisites

You will need accounts on:

| Service | Purpose | Sign up |
|---|---|---|
| GitHub | Source hosting + CI trigger for Netlify | https://github.com/signup |
| Netlify | Static site + serverless functions hosting | https://app.netlify.com/signup |
| Supabase | Postgres + auth | https://supabase.com/dashboard/sign-up |
| Stripe | Billing | https://dashboard.stripe.com/register |
| Anthropic Console | AI API access | https://console.anthropic.com/ |
| (Optional) Domain registrar | Custom domain | Whoever holds your domain |

Local tools required:

- Node.js 20+ and npm 10+
- `git`
- (Optional) Netlify CLI: `npm install -g netlify-cli`
- (Optional) Supabase CLI: `npm install -g supabase`
- (Optional) Stripe CLI for testing webhooks locally: https://stripe.com/docs/stripe-cli

Verify a clean local build before continuing:

```
cd create-a-web-app-that-can
npm install
npx tsc --noEmit
npm run lint
npm test
npx vite build --configLoader runner
```

If any of those fail, fix them before going further. They will fail again on Netlify and you will waste a deploy slot tracking it down there.

---

# Phase 1 — Source control

The repo is not yet initialized as a git repo. Initialize it and push to a GitHub repository so Netlify can build from it.

1. From `create-a-web-app-that-can/`:

```
sh git init -b main git add -A git commit -m "Initial commit: web-launch-guard"
```

1. Create a new empty GitHub repository (private is fine) and copy the SSH or HTTPS URL it gives you.
2. Add the remote and push:

```
sh git remote add origin git@github.com:<your-org>/<repo>.git git push -u origin main
```

1. Confirm `.gitignore` covers `node_modules/`, `dist/`, `.env*`, `playwright-browsers/`, and `test-results/`. The repo's existing `.gitignore` should already cover these — double check before pushing secrets.

---

# Phase 2 — Provision Supabase

## 2.1 Create the project

1. Open https://supabase.com/dashboard/new.
2. Create a new project. Pick a region close to your users (Anthropic API is US-hosted; if you expect mostly US/EU traffic, `us-east-1` keeps the round trip from Netlify functions short).
3. Set a strong database password and save it in your password manager. You will not need it for the app, but you will need it for the SQL editor if you ever connect directly.
4. Wait for the project to finish provisioning (1–2 minutes).

## 2.2 Capture the keys you will need

In Project Settings → API, copy:

- **Project URL** → `SUPABASE_URL` and `VITE_SUPABASE_URL`
- `anon` `public` **key** → `VITE_SUPABASE_ANON_KEY`
- `service_role` `secret` **key** → `SUPABASE_SERVICE_ROLE_KEY`

The `service_role` key bypasses RLS. Never put it in `VITE_*` env vars or expose it to the browser. It belongs only on Netlify functions.

## 2.3 Apply migrations in order

The repo ships four migrations under `supabase/migrations/`:

| File | What it does |
|---|---|
| 0001_initial_schema.sql | Tables, indexes, RLS for every domain, scan job, finding, report, subscription. |
| 0002_add_stripe_subscription_fields.sql | Adds Stripe customer/subscription/price/billing-interval columns. |
| 0003_tiered_billing_and_quota.sql | Tier model (basic/pro/enterprise), per-domain subscription FK, server-issued verification tokens, run-quota and rate-limit tables, broader findings categories. |
| 0004_security_hardening.sql | Atomic reserve_scan_slot() function with advisory locks, partial unique index on verified hostnames, drops authenticated INSERT/UPDATE policies on reports + findings (service-role only). |

Apply them in number order. Two ways:

**Option A — SQL editor (simplest):** in Supabase dashboard → SQL Editor → New query, paste the file content, click Run. Repeat for all four files in order.

**Option B — Supabase CLI:**

```
supabase link --project-ref <your-ref>
supabase db push
```

Verify after applying:

```
select count(*) from pg_proc where proname = 'reserve_scan_slot';
-- expect: 1
select count(*) from pg_indexes where indexname = 'domains_verified_hostname_idx';
-- expect: 1
```

## 2.4 Configure auth

In Authentication → Providers:

- **Email** is enabled by default. Decide whether to require email confirmation (recommended for production). If enabled, Supabase will need outbound SMTP — either use the dev SMTP (rate-limited, fine for low volume) or wire up Postmark/Resend in Auth → Email Templates.
- **Google** (optional): create OAuth client at https://console.cloud.google.com/apis/credentials, add `https://<your-supabase-project>.supabase.co/auth/v1/callback` as the authorized redirect, paste client ID/secret into Supabase.

In Authentication → URL Configuration:

- **Site URL**: leave blank for now. Set it to the Netlify URL after Phase 5.
- **Additional Redirect URLs**: same — fill in after Phase 5.

---

# Phase 3 — Provision Stripe (test mode first)

Always do the full setup in Stripe **test mode** first. After you have verified an end-to-end purchase works, repeat the same setup in live mode (see Phase 10).

## 3.1 Create the three products

In Stripe Dashboard → Product catalog → Add product, create:

| Product name | Description |
|---|---|
| Web Launch Guard Basic | One verified domain, 5 scans per month, SOC 2 checklist. |
| Web Launch Guard Pro | Up to 3 verified domains, 20 scans per domain per month, AI review and controlled live inspection. |
| Web Launch Guard Enterprise | Up to 10 verified domains, 50 scans per domain per month, AI review and controlled live inspection. |

## 3.2 Create six prices

Each product needs a monthly recurring price and an annual recurring price.

| Product | Price | Interval | Env var to populate |
|---|---|---|---|
| Basic | $29.00 | Monthly | STRIPE_BASIC_MONTHLY_PRICE_ID |
| Basic | $228.00 | Yearly ($19/mo × 12) | STRIPE_BASIC_ANNUAL_PRICE_ID |
| Pro | $129.00 | Monthly | STRIPE_PRO_MONTHLY_PRICE_ID |
| Pro | $1188.00 | Yearly ($99/mo × 12) | STRIPE_PRO_ANNUAL_PRICE_ID |
| Enterprise | $399.00 | Monthly | STRIPE_ENTERPRISE_MONTHLY_PRICE_ID |
| Enterprise | $4188.00 | Yearly ($349/mo × 12) | STRIPE_ENTERPRISE_ANNUAL_PRICE_ID |

Click into each price after creating, copy the `price_...` ID, paste into a notes file or a password manager — you will paste these into Netlify in Phase 5.

## 3.3 Capture the API key

Stripe Dashboard → Developers → API keys → reveal the **Secret key** (`sk_test_...` in test mode, `sk_live_...` in live mode). This becomes `STRIPE_SECRET_KEY` on Netlify.

## 3.4 Configure the Customer Portal

Stripe Dashboard → Settings → Billing → Customer Portal:

- **Functionality**: enable "Update payment method", "Cancel subscriptions", and "Update subscriptions" if you want to allow plan changes.
- **Cancellation policy**: pick "Immediately" or "At end of billing period" per your product policy.
- **Branding**: set the business name and (optional) logo.
- Click Save.

The webhook setup (Phase 6) needs a public URL, so park that for now.

---

# Phase 4 — Provision Anthropic

1. Sign in at https://console.anthropic.com.
2. Add a payment method in Plans & Billing — the API requires usage credit on file before keys produce successful responses.
3. Settings → API Keys → Create key. Name it something like `web-launch-guard-prod`. Copy the value (`sk-ant-...`) immediately — you will not see it again.
4. The default model used by the app is `claude-sonnet-4-6`. You can override per environment via the `ANTHROPIC_MODEL` env var.

---

# Phase 5 — First deploy to Netlify

The repo already includes a `netlify.toml` that points at the right build command and functions directory.

## 5.1 Connect the repo

1. Netlify dashboard → Add new site → Import an existing project.
2. Authorize GitHub, pick the repository you pushed in Phase 1.
3. Build settings (Netlify usually auto-detects): - **Build command**: `npm run build` - **Publish directory**: `dist` - **Functions directory**: `netlify/functions` - **Node version**: 20

## 5.2 Configure environment variables

Site settings → Environment variables → Add a variable. Add **all** of these:

| Key | Source |
|---|---|
| VITE_SUPABASE_URL | Phase 2.2 |
| VITE_SUPABASE_ANON_KEY | Phase 2.2 |
| SUPABASE_URL | Phase 2.2 |

| Key | Source |
|---|---|
| SUPABASE_SERVICE_ROLE_KEY | Phase 2.2 |
| STRIPE_SECRET_KEY | Phase 3.3 (sk_test_... for now) |
| STRIPE_BASIC_MONTHLY_PRICE_ID | Phase 3.2 |
| STRIPE_BASIC_ANNUAL_PRICE_ID | Phase 3.2 |
| STRIPE_PRO_MONTHLY_PRICE_ID | Phase 3.2 |
| STRIPE_PRO_ANNUAL_PRICE_ID | Phase 3.2 |
| STRIPE_ENTERPRISE_MONTHLY_PRICE_ID | Phase 3.2 |
| STRIPE_ENTERPRISE_ANNUAL_PRICE_ID | Phase 3.2 |
| ANTHROPIC_API_KEY | Phase 4 |
| ANTHROPIC_MODEL | claude-sonnet-4-6 (or override) |
| NODE_ENV | production |

Two values come later:

- `STRIPE_WEBHOOK_SECRET` — Phase 6.
- `URL` — Netlify auto-populates this on every build context. You can leave it alone, but if you want to pin it explicitly, set it to your custom domain after Phase 7.

Set the scope to **All deploy contexts** unless you intend to use a separate preview/staging Stripe account.

### 5.3 Trigger the first deploy

Netlify will deploy automatically once you save env vars (or click Deploys → Trigger deploy → Deploy site). Watch the build log:

- TypeScript should compile cleanly.
- Vite should produce `dist/` output around 400 KB.
- The functions runtime should bundle the seven files in `netlify/functions/`.

If the build fails, the most common causes are:

- Missing env var the build process needs (only the `VITE_*` ones are consumed at build time; the rest are runtime-only).
- TypeScript errors that you skipped locally — fix and push.

When the deploy succeeds, copy your site URL (something like `https://wlg-prod.netlify.app`). Save it.

---

## Phase 6 — Wire up webhooks and redirects

Now that you have a public URL, finish the half-configured pieces.

### 6.1 Supabase — Site URL and redirects

Supabase dashboard → Authentication → URL Configuration:

- **Site URL**: set to your Netlify URL (or your custom domain — see Phase 7 if you have one ready).
- **Additional Redirect URLs**: add the same URL plus a wildcard for hash routes, e.g.:
- `https://wlg-prod.netlify.app`
- `https://wlg-prod.netlify.app/**`

This is what makes email confirmation links and OAuth callbacks land back on your app instead of `localhost`.

### 6.2 Stripe — webhook endpoint

Stripe Dashboard → Developers → Webhooks → Add endpoint:

- **Endpoint URL**: `https://<your-netlify-site>/.netlify/functions/stripe-webhook`
- **Events to send** (select these and only these):
- `customer.subscription.created`
- `customer.subscription.updated`
- `customer.subscription.deleted`
- Click Add endpoint.

On the resulting endpoint detail page, click **Reveal signing secret** and copy `whsec_...`.

Back in Netlify → Site settings → Environment variables:

- Add `STRIPE_WEBHOOK_SECRET` = the `whsec_...` you just copied.
- Trigger a redeploy (Deploys → Trigger deploy → Clear cache and deploy).

### 6.3 Send a test event

Stripe Dashboard → Webhooks → your endpoint → Send test webhook → pick `customer.subscription.created` → Send.

You should see HTTP 200 in the delivery history. If you see 400, the signing secret is wrong; if 500, the function failed — check Netlify → Functions → `stripe-webhook` logs.

---

## Phase 7 — Custom domain (optional but recommended)

If you are launching on a custom domain (e.g. `app.ctfdigital.store`):

1. Netlify → Domain management → Add a domain → enter the hostname.

2. Netlify shows you DNS records. Either: - **Recommended**: switch your domain's nameservers to Netlify DNS so Netlify manages the apex + subdomain records automatically. - **Alternative**: keep your existing DNS and add the records Netlify prints (typically one CNAME or A record).
3. Wait for DNS to propagate (5–30 minutes). Netlify will issue a Let's Encrypt cert automatically once it can resolve the hostname.
4. Once the cert is live, set the custom domain as the **primary domain** in Netlify so all `*.netlify.app` URLs redirect to it.
5. Update the `URL` env var in Netlify (or just leave it — Netlify pins `URL` to the primary domain by default on builds), then redeploy.
6. Repeat steps 6.1 and 6.2 with the new origin: - Supabase Site URL = the custom domain. - Stripe webhook URL = `https://<custom-domain>/.netlify/functions/stripe-webhook`. Add the new endpoint, copy its signing secret to `STRIPE_WEBHOOK_SECRET`, redeploy, then **disable** the old `*.netlify.app` endpoint in Stripe.

---

## Phase 8 — Smoke test the live flow

Use **Stripe test mode** for this. The product copy in the app does not yet distinguish test from live, so test in test mode and switch to live only after you confirm everything works.

You will need:

- A throwaway email you can receive mail at (for signup confirmation).
- A domain whose DNS you control (for the verification step). Stripe test cards do not need this, but the per-domain scan flow requires a real DNS record at a real public host.
- Stripe test card: `4242 4242 4242 4242`, any future expiry, any 3-digit CVC, any ZIP.

End-to-end happy path:

1. Visit the production URL. Click **Sign up**.
2. Sign up with email + password. Confirm via the email link.
3. On the dashboard, scroll to **Add a subscription**. Pick **Basic monthly**. Stripe Checkout opens.
4. Pay with the test card. You should land on `/dashboard?billing=success`.
5. Within ~1–2 seconds, Stripe fires the `customer.subscription.created` webhook. The dashboard should refresh to show one Basic subscription. (If it does not, hard-refresh — the client-side cache is naive.)
6. **Add a domain**: pick a real domain you own. Enter the hostname (e.g. `example.com`). Choose the Basic subscription you just bought. Click Add.
7. The new domain row appears with a TXT record. Publish that TXT record in your DNS: - Name: `_weblaunchguard.example.com` (or apex, both are checked) - Value: `weblaunchguard-verify=wlguard_<random hex>` - TTL: 60 (low while you test)
8. Wait 1–5 minutes for DNS to propagate. Click **Check DNS**. The status flips to `verified`.

9. **Run a scan**: pick the verified domain in the scan dropdown, paste `https://example.com`, click Scan site.
10. Findings + SOC 2 checklist should render. Quota should show `1/5`.
11. Click into a saved report. Click **Export PDF**. A `web-launch-guard-<id>.pdf` should download with content matching the finding list.
12. **Upgrade**: buy a Pro or Enterprise subscription. After the new subscription appears, attach a domain to it. Run **AI analysis** on the dashboard — Claude should return findings within a few seconds.
13. **Manage billing**: click **Manage billing** → Stripe Customer Portal opens. Verify you can update payment method and cancel.

If any step fails, see Phase 11 (Troubleshooting).

---

# Phase 9 — Switch to Stripe live mode

Once test mode works end-to-end:

1. In the Stripe dashboard, toggle from **Test mode** to **Live mode**.
2. Repeat Phase 3.1, 3.2, 3.3, 3.4 in live mode. The price IDs will be different (`price_live_...`) — you must capture the new ones.
3. Repeat Phase 6.2 in live mode. The webhook endpoint URL stays the same but the signing secret is different (`whsec_live_...`).
4. In Netlify, update these env vars to their live values: - `STRIPE_SECRET_KEY` → `sk_live_...` - `STRIPE_WEBHOOK_SECRET` → live `whsec_...` - All six `STRIPE_*_PRICE_ID` → live IDs
5. Trigger a redeploy.
6. Run Phase 8 again with a real card (or a Stripe-issued test card in live mode if your account is set up for that). Cancel the subscription immediately afterward to avoid charging yourself.

---

# Phase 10 — Operations and monitoring

## 10.1 Logs and dashboards

Bookmark these:

- **Netlify Functions logs**: Site → Functions → click each function name to see invocation logs and errors. The `console.error` calls added to `passive-scan`, `pro-analysis`, `create-checkout`, `stripe-webhook`, and `rate-limit` surface here.
- **Supabase logs**: Project → Logs → choose "Postgres logs" for query errors, "Auth logs" for sign-in failures, "Edge function logs" if you later move any code there.
- **Stripe webhook deliveries**: Webhooks → your endpoint → Recent deliveries. Failed deliveries show the request and response.

- **Anthropic usage**: Console → Plans & Billing → Usage. Set a monthly spend limit while you tune the AI flow so a runaway loop cannot drain the account.

### 10.2 Recommended alerts

- Stripe: enable email alerts for failed payments and disputed charges.
- Anthropic: set a usage threshold alert.
- Netlify: enable deploy failure notifications.
- (Optional) Add a third-party uptime monitor (Better Stack, Uptime Robot) hitting your homepage every minute.

### 10.3 Routine tasks

- **Database backups**: Supabase Free / Pro plans include daily PITR. For Free, keep a periodic `pg_dump` if data loss matters.
- **API key rotation**: rotate `STRIPE_SECRET_KEY`, `ANTHROPIC_API_KEY`, and `SUPABASE_SERVICE_ROLE_KEY` at least annually or any time a contributor leaves the project.
- **Migration discipline**: never edit a previously-applied migration. Add a new file (`0005_*.sql`) and apply it the same way.

### 10.4 Capacity notes

- Netlify Functions on the Free plan: 125k invocations/month, 100h runtime. Each scan is one passive-scan invocation; AI scans add a pro-analysis invocation. Above ~3000 scans/month consider upgrading.
- Supabase Free plan: 500 MB database, 2 GB egress, paused after 1 week of inactivity. Move to Pro before launching to a real audience.
- Anthropic: Sonnet 4.6 is around $3/$15 per million input/output tokens. Each AI scan uses on the order of 10–20K tokens; budget accordingly.

---

## Phase 11 — Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| Build fails on Netlify, "Missing VITE_SUPABASE_URL" | Env var not set, or set in wrong scope | Site settings → Environment variables, ensure scoped to "All contexts", redeploy. |
| Sign-up email never arrives | Default Supabase SMTP rate-limited; spam folder | Configure custom SMTP in Authentication → Email Templates. |
| OAuth redirects to localhost | Site URL still set to dev value | Phase 6.1 — set production Site URL and redirect URLs. |

| Symptom | Likely cause | Fix |
|---|---|---|
| Checkout returns "Checkout session creation failed" | One of the six STRIPE_*_PRICE_ID env vars missing | Netlify Functions logs → create-checkout will show which env var is missing. Add it, redeploy. |
| Stripe webhook 400 | Signing secret mismatch | Confirm STRIPE_WEBHOOK_SECRET matches the one in Stripe → Webhooks → Signing secret. Watch out for trailing whitespace when pasting. |
| Subscription paid but app still shows "no active subscriptions" | Webhook didn't fire, or fired but failed | Stripe → Webhooks → your endpoint → Recent deliveries. If 200 but app doesn't show it, check that metadata.userId is set on the subscription (looking at the Stripe subscription object). |
| Verify domain ownership before scanning after publishing TXT | DNS propagation lag | Wait 2–10 min, then click Check DNS again. Verify with dig TXT _weblaunchguard.example.com from a terminal. |
| Run quota exhausted | Used all monthly slots | Wait until next billing period, or upgrade tier. Quota resets on Stripe current_period_start. |
| AI request returns 503 | ANTHROPIC_API_KEY missing or invalid | Check the env var. The error message names the missing key explicitly. |
| AI analysis requires the Pro or Enterprise tier on Basic | Working as designed | Upgrade to Pro or Enterprise. |
| Rate limit 429 immediately after a single request | Rate limiter failed closed on a DB error | Check Supabase Postgres logs for query errors on rate_limits or prune_rate_limits. The limiter intentionally fails closed; transient DB issues become 429s. |
| Report PDF is blank | The report's payload JSONB column is empty | Look at the saved row in Supabase. If findings is empty, the scan returned no findings (the page is well-configured). If something is wrong, the function logs will show. |

For anything not in this table, the first stop is always **Netlify** → **Functions** → **\<the function involved>** → **Logs**. Every function in this codebase calls `console.error` with structured context on failure.

---

# Quick reference

## Env var checklist (Netlify)

```
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
STRIPE_SECRET_KEY
STRIPE_WEBHOOK_SECRET
STRIPE_BASIC_MONTHLY_PRICE_ID
STRIPE_BASIC_ANNUAL_PRICE_ID
STRIPE_PRO_MONTHLY_PRICE_ID
STRIPE_PRO_ANNUAL_PRICE_ID
STRIPE_ENTERPRISE_MONTHLY_PRICE_ID
STRIPE_ENTERPRISE_ANNUAL_PRICE_ID
ANTHROPIC_API_KEY
ANTHROPIC_MODEL=claude-sonnet-4-6
NODE_ENV=production
```

`URL` is auto-populated by Netlify; only set it manually if you need to override the default.

## Webhook events Stripe needs to send

```
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
```

## Stripe test cards

```
Success            4242 4242 4242 4242
Requires 3DS auth  4000 0027 6000 3184
Declined           4000 0000 0000 0002
```

Any future expiry, any 3-digit CVC, any ZIP.